

# COMP3131/9102: Programming Languages and Compilers

*Jingling Xue*

School of Computer Science and Engineering  
The University of New South Wales  
Sydney, NSW 2052, Australia

<http://www.cse.unsw.edu.au/~cs3131>

<http://www.cse.unsw.edu.au/~cs9102>

Copyright ©2024, Jingling Xue

## Week 3 (1st Lecture): Top-Down Parsing (Revisited)

- Feedback for Assignment 1
- Revisit First and Follow sets
- Revisit Top-Down Parsing
- Formal Grammars
- Equivalence between Regular Grammars and FAs

## A Simple Tool for Computing First and Follow Sets

<https://gist.github.com/DmitrySoshnikov/924ceefb1784b30c5ca6>

## One More Example to Explain Why LL(1) Parsers Cannot Handle Left Recursion

## Left Recursion Again

- A non-LL(1) grammar (with left recursion):

$$\begin{array}{l} S \rightarrow S + i \\ \quad | i \end{array}$$

- A LL(1) grammar (without left recursion):

$$\begin{array}{l} S \rightarrow iT \\ T \rightarrow +iT \mid \epsilon \end{array}$$

$$\begin{array}{lll} \text{Select}(S \rightarrow iT) & = \text{First}(iT) & = \{i\} \\ \text{Select}(T \rightarrow +iT) & = \text{First}(+iT) & = \{+\} \\ \text{Select}(T \rightarrow \epsilon) & = \text{First}(\epsilon) \setminus \{\epsilon\} \cup \text{Follow}(T) & = \{\$\} \end{array}$$

## LL(1) Parser for the Grammar Without Left Recursion

```

public void parseGoal() {
    parseS();
    if (currentToken == '$')
        System.out.println("The sentence is grammatical");
    else
        System.out.println("The sentence is NOT grammatical");
}
void parseS() {
    match('i');
    parseT();
}
void parseT() {
    if (currentToken == '+') {
        match('+');
        match('i');
        parseT();
    } else if (currentToken == '$') {
        ; // do nothing
    } else {
        System.out.println("The sentence is NOT grammatical");
        System.exit(0);
    }
}

```

Add a new start symbol *Goal* by adding  $Goal \rightarrow S$  to make sure *Goal* never appears anywhere else

## LL(1) Parser Simplified

```
public void parseGoal() {  
    parseS();  
    if (currentToken == '$')  
        System.out.println("The sentence is grammatical");  
    else  
        System.out.println("The sentence is NOT grammatical");  
}  
void parseS() {  
    match('i');  
    parseT();  
}  
void parseT() {  
    if (currentToken == '+') {  
        match('+');  
        match('i');  
        parseT();  
    }  
}
```

## LL( $k$ ) Grammar and Parsing

- A grammar is LL( $k$ ) if it can be parsed **deterministically** using  $k$  tokens of lookahead
- A formal definition for LL( $k$ ) grammars can be found in Grune and Jacobs' book  
[https://dickgrune.com/Books/PTAPG\\_1st\\_Edition/](https://dickgrune.com/Books/PTAPG_1st_Edition/)
- Grammar 1 in Slide 172 is not LL( $k$ ) for any  $k$ !
- However, Grammar 2 in Slide 172 is LL(1)
- Only a understanding of LL(1) is required this year



## The VC Grammar Is Not LL(1)

- *Program*: common prefix in its production right-hand sides
- A lots of left-recursive productions

Must eliminate both parsing conflicts to write your recogniser for Assignment 2

## Formal Grammar

A grammar  $G$  is a quadruple  $(V_T, V_N, S, P)$ , where

- $V_T$ : a finite set of terminal symbols or **tokens**
- $V_N$ : a finite set of nonterminal symbols ( $V_T \cap V_N = \emptyset$ )
- $S$ : a unique start symbol ( $S \in V_N$ )
- $P$ : a finite set of rules or productions of the form:

$$\alpha \rightarrow \beta \quad (\alpha \neq \epsilon)$$

- $\alpha$  is a string of **one** or more terminals and nonterminals
- $\beta$  is a string of **zero** or more terminals and nonterminals

## Chomsky's Hierarchy

Depending on  $\alpha \rightarrow \beta$ , four types of grammars distinguished:

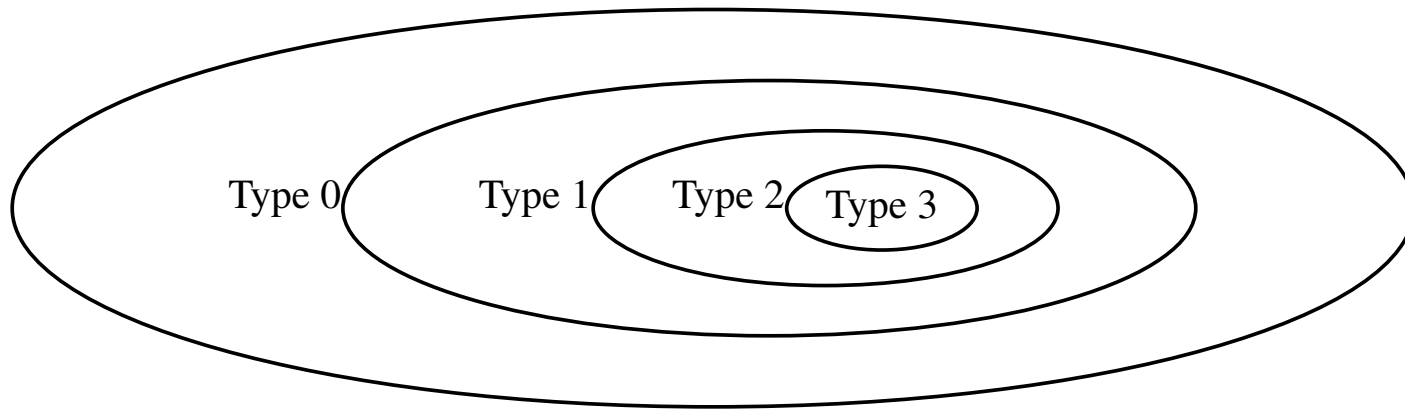
| GRAMMAR | KNOWN AS                                 | DEFINITION                | MACHINE                     |
|---------|------------------------------------------|---------------------------|-----------------------------|
| Type 0  | phrase-structure grammar                 | $\alpha \neq \epsilon$    | Turing machine              |
| Type 1  | context-sensitive grammar<br>CSGs        | $ \alpha  \leq  \beta $   | linear bounded<br>automaton |
| Type 2  | context-free grammar<br>CFGs             | $A \rightarrow \alpha$    | stack automaton             |
| Type 3  | right-linear grammar<br>regular grammars | $A \rightarrow a \mid aB$ | finite automaton            |

### Note:

- $a$  is a terminal.
- regular grammars can also be specified by left-linear grammars:

$$A \rightarrow a \mid Ba$$

## Relationships between the Four Types of Languages



- Type  $k$  language is a proper subset of Type  $k - 1$  language.
- The existence of a Type 0 language is proved:

page 228, J. Hopcroft and J. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Addison-Wesley, 1979.

## Regular Expressions, Regular Grammars and Finite Automata

- All three are **equivalent**:

- Example:

– Regular expression:  $[A - Z a - z _][A - Z a - z 0 - 9 _]^*$

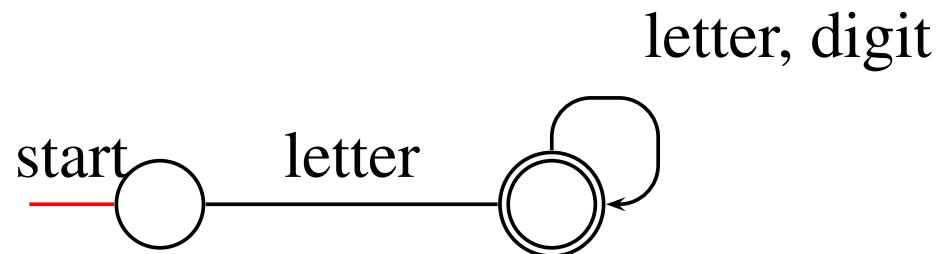
– Regular grammar:

*identifier*  $\rightarrow$  *letter* | *identifier letter* | *identifier digit*

*letter*  $\rightarrow$  A | B | ... | Z | a | b | ... | z | \_

*digit*  $\rightarrow$  0 | 1 | ... | 9

– DFA:



## Limitations of Regular Grammars

- Cannot generate **nested** constructs
- The following language is not regular

$$L = \{a^n b^n \mid n \geq 0\}$$

- But  $L$  is context-free:  $S \rightarrow \epsilon \mid aSb$
- Regular grammars (expressions) powerful enough for specifying tokens, which are not nested
- By replacing “ $a$ ” and “ $b$ ” with “(” and “)”, the following

$$L = \{( )^n \mid n \geq 0\}$$

is not regular

- Formal proof: Pages 180 – 181 of Red / §4.2.7 of Purple
- **Regular grammars (finite automata) cannot count**

## Limitations of CFGs

- CFLs only include a subset of all languages
- Examples of non-CFL constructs:
  - An abstraction of variable declaration before use:

$$L_1 = \{wcw \mid w \text{ is in } (a|b)^*\}$$

where the 1st  $w$  represents a declaration and the 2nd its use

- a method called with the right number of arguments:

$$L_2 = \{a^n b^m c^n d^m \mid n \geq 1, m \geq 1\}$$

where  $a^n$  and  $b^m$  represent formal parameter lists in two methods with  $n$  and  $m$  arguments, respectively, and  $c^n$  and  $d^m$  represent actual parameter lists in two calls to the two methods.

- Can count two but not three:

$$L_3 = \{a^n b^n c^n \mid n \geq 0\}$$

## Limitations of CFGs (Cont'd)

- $L_3$  is **not** context-free

- The language:

$$L_3 = \{a^n b^n c^n \mid n \geq 0\}$$

- The grammar:

- A **Context-Sensitive Grammar** (CSG) (that is not a CFG) for  $L_3$ :

| CSG: |               |        | A derivation for $aabbcc$ |               |           |
|------|---------------|--------|---------------------------|---------------|-----------|
| $S$  | $\rightarrow$ | $aSBC$ | $S$                       | $\Rightarrow$ | $aSBC$    |
| $S$  | $\rightarrow$ | $abC$  |                           | $\Rightarrow$ | $aabCBC$  |
| $CB$ | $\rightarrow$ | $BC$   |                           | $\Rightarrow$ | $aabBCC$  |
| $bB$ | $\rightarrow$ | $bb$   |                           | $\Rightarrow$ | $aabbCC$  |
| $bC$ | $\rightarrow$ | $bc$   |                           | $\Rightarrow$ | $aabbccC$ |
| $cC$ | $\rightarrow$ | $cc$   |                           | $\Rightarrow$ | $aabbcc$  |



## Why CFGs in Parser Construction?

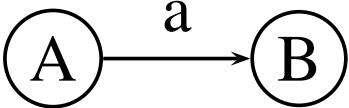
- Types 0 and 1 are less understood, no simple ways of constructing parsers for them, and parsers for these languages are slow
- Type 3 cannot define recursive language constructs
- **Type 2 – context-free grammars (CFGs):**
  - Easily related to the structure of the language; productions give us a good idea of what to expect in the language
  - Close relationships between the productions and the corresponding computations, which is the basis of **syntax-directed translation**
  - Efficient parsers can be built automatically from CFGs

## Equivalence between Regular Grammars and FAs

- Week 1 (2nd Lecture): the equivalence among REs and FAs
- Slides 228 – 233: NFAs  $\equiv$  Regular Grammars

## Converting NFAs to Right-Linear Grammars

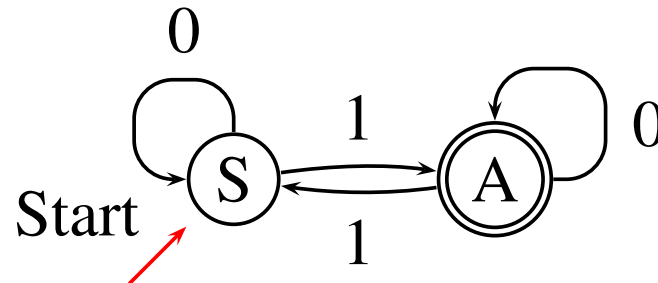
- The alphabet: the same
- For each state in the NFA, create a nonterminal with the same name.
- The start state will be the start symbol
- Then

| TRANSITION                                                                          | PRODUCTION                           |
|-------------------------------------------------------------------------------------|--------------------------------------|
|  | $\Rightarrow A \rightarrow aB$       |
|  | $\Rightarrow A \rightarrow \epsilon$ |

where  $a \in \Sigma$  or  $a = \epsilon$

## Example 1

- The DFA:



- The grammar:

$$S \rightarrow 0S$$

$$S \rightarrow 1A$$

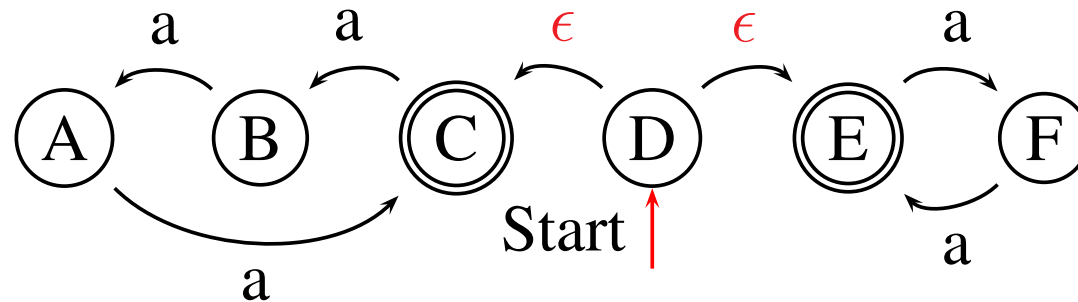
$$A \rightarrow 0A$$

$$A \rightarrow 1S$$

$$A \rightarrow \epsilon$$

## Example 2

- The NFA:



- The grammar:

$$\begin{array}{ll}
 D \rightarrow C & A \rightarrow aC \\
 D \rightarrow E & E \rightarrow aF \\
 C \rightarrow aB & E \rightarrow \epsilon \\
 C \rightarrow \epsilon & F \rightarrow aE \\
 B \rightarrow aA &
 \end{array}$$

## Converting Right-Linear Grammars to NFAs

- The alphabet: the same
- For each nonterminal, create a state in the NFA with the same name. The start symbol will be the start state
- Add one new state and make it the **only** final state  $\mathcal{F}$
- Then

| PRODUCTION         | TRANSITION    |                                                                                         |
|--------------------|---------------|-----------------------------------------------------------------------------------------|
| $A \rightarrow aB$ | $\Rightarrow$ | $\textcircled{A} \xrightarrow{a} \textcircled{B} \quad T(A, a) = B$                     |
| $A \rightarrow a$  | $\Rightarrow$ | $\textcircled{A} \xrightarrow{a} \textcircled{\mathcal{F}} \quad T(A, a) = \mathcal{F}$ |

where  $a \in \Sigma$  or  $a = \epsilon$

## Example 1

- The grammar:

$$S \rightarrow 0S$$

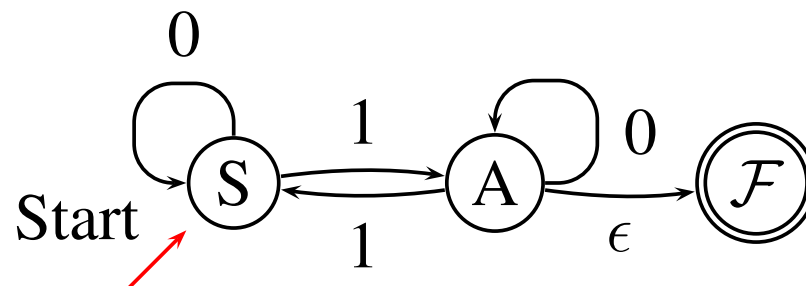
$$S \rightarrow 1A$$

$$A \rightarrow 0A$$

$$A \rightarrow 1S$$

$$A \rightarrow \epsilon$$

- The NFA:



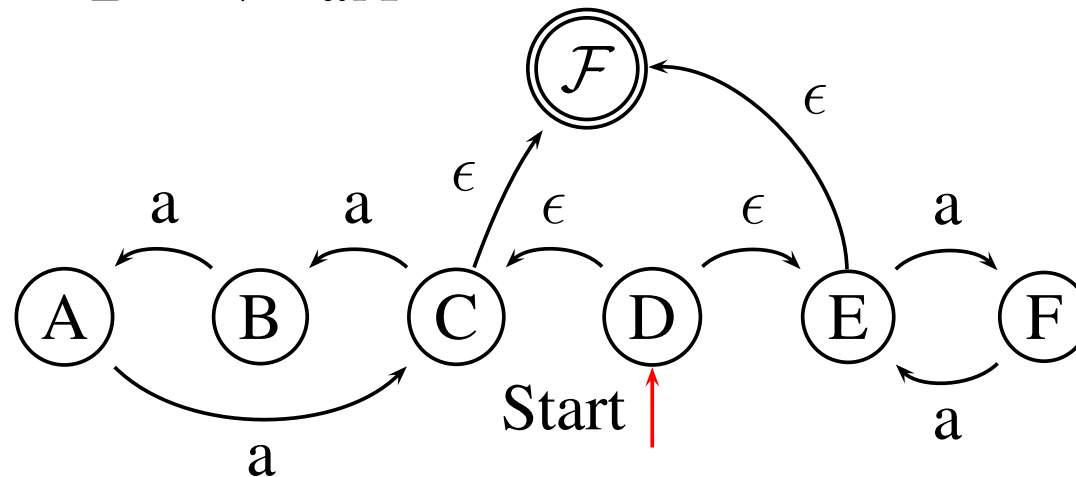
- This NFA accepts the same language as the one in Slide 229

## Example 2

- The grammar:

$$\begin{array}{ll}
 D \rightarrow C & A \rightarrow aC \\
 D \rightarrow E & E \rightarrow aF \\
 C \rightarrow aB & E \rightarrow \epsilon \\
 C \rightarrow \epsilon & F \rightarrow aE \\
 B \rightarrow aA
 \end{array}$$

- The NFA:



- This NFA accepts the same language as the NFA in Slide 230



## Reading

- Many online material on computing First and Follow sets
- The Dragon textbook
- Optional:
  - Conversion of Left Linear Grammars to NFAs:  
<https://scanftree.com/automata/conversion-of-left-linear-grammar-to-finite-automata>
  - Conversion of left linear grammars to right linear grammars:  
<https://www.tutorialspoint.com/how-to-convert-left-linear-grammar-to-right-linear-grammar>

**Next Class:** Abstract Syntax Trees (Preparing You for Assignment 3)